

Case Study: Train Reservation System – Passenger Management

Technology Stack: Spring MVC + Hibernate + JSP + MySQL

1. Project Title

Train Reservation System – Passenger Management Module

2. Problem Statement

Indian Railways wants to digitize passenger management to eliminate manual paperwork and improve reservation efficiency.

Develop a Passenger Management Module using Spring MVC, Hibernate, JSP, and MySQL that allows railway staff to perform complete CRUD operations on passenger information. The application should follow the MVC architecture using annotation-based configuration.

3. Objective

Develop a web application to manage passenger records where users can add, view, search, update, and delete passenger details using Spring MVC, Hibernate, JSP, JSTL, MySQL, and Maven.

4. Modules

- Add Passenger
- Display Passenger List
- Search Passenger
- Update Passenger
- Delete Passenger

5. Functional Requirements

Add Passenger, View Passengers, Update Passenger, Delete Passenger, Search Passenger by ID.

6. Non-Functional Requirements

Responsive UI, Data Validation, MVC Architecture, Hibernate ORM, Annotation Configuration, Exception Handling.

7. Passenger Entity

passengerId, passengerName, age, gender, mobile, email, aadhaarNumber, sourceStation, destinationStation, journeyDate, seatClass, ticketFare.

8. Database Table

Table: passenger

Columns:

passenger_id, passenger_name, age, gender, mobile, email, aadhaar_number, source_station, destination_station, journey_date, seat_class, ticket_fare

9. Project Folder Structure

TrainReservationSystem

src/main/java

controller/

dao/

entity/

service/

config/

src/main/webapp

WEB-INF/views/

home.jsp

addPassenger.jsp

editPassenger.jsp

passengerList.jsp

searchPassenger.jsp

resources/css/

pom.xml

10. Application Flow

Home → Add Passenger → Save Passenger → View Passenger List → Update Passenger → Delete Passenger → Search Passenger

11. Screen Designs

Home: Add Passenger | View Passengers | Search Passenger

Add Passenger Form: Name, Age, Gender, Mobile, Email, Aadhaar, Source, Destination, Journey Date, Seat Class, Fare.

Passenger List: Table with Edit/Delete actions.

Search Passenger: Search by Passenger ID.

Edit Passenger: Update journey and contact details.

12. Business Rules

Age > 5 years; Mobile must be 10 digits; Aadhaar must be 12 digits; Fare > 0; Journey date cannot be in the past; Source and Destination cannot be the same; Email unique; Passenger ID auto-generated.

13. Expected Learning Outcomes

Spring MVC, DispatcherServlet, Controllers, JSP Forms, Hibernate ORM, SessionFactory, CRUD Operations, Dependency Injection, Transaction Management, Maven, JSTL, MVC Pattern.

14. Suggested SQL

```
CREATE TABLE passenger(  
  
    passenger_id INT PRIMARY KEY AUTO_INCREMENT,  
  
    passenger_name VARCHAR(100),  
  
    age INT,  
  
    gender VARCHAR(10),  
  
    mobile VARCHAR(15),  
  
    email VARCHAR(100) UNIQUE,  
  
    aadhaar_number VARCHAR(20),  
  
    source_station VARCHAR(50),  
  
    destination_station VARCHAR(50),  
  
    journey_date DATE,  
  
    seat_class VARCHAR(30),
```

ticket_fare DOUBLE

);

15. Suggested URLs

/ , /passenger/add, /passenger/save, /passenger/list, /passenger/edit/{id},
/passenger/update, /passenger/delete/{id}, /passenger/search

16. Future Enhancements

Train Management, Station Management, Ticket Booking, Seat Availability, Fare Calculation, PNR Status, Registration & Login, Online Payment, Ticket Cancellation, Notifications, Admin Dashboard, Reports.